

Collaborating with GitHub

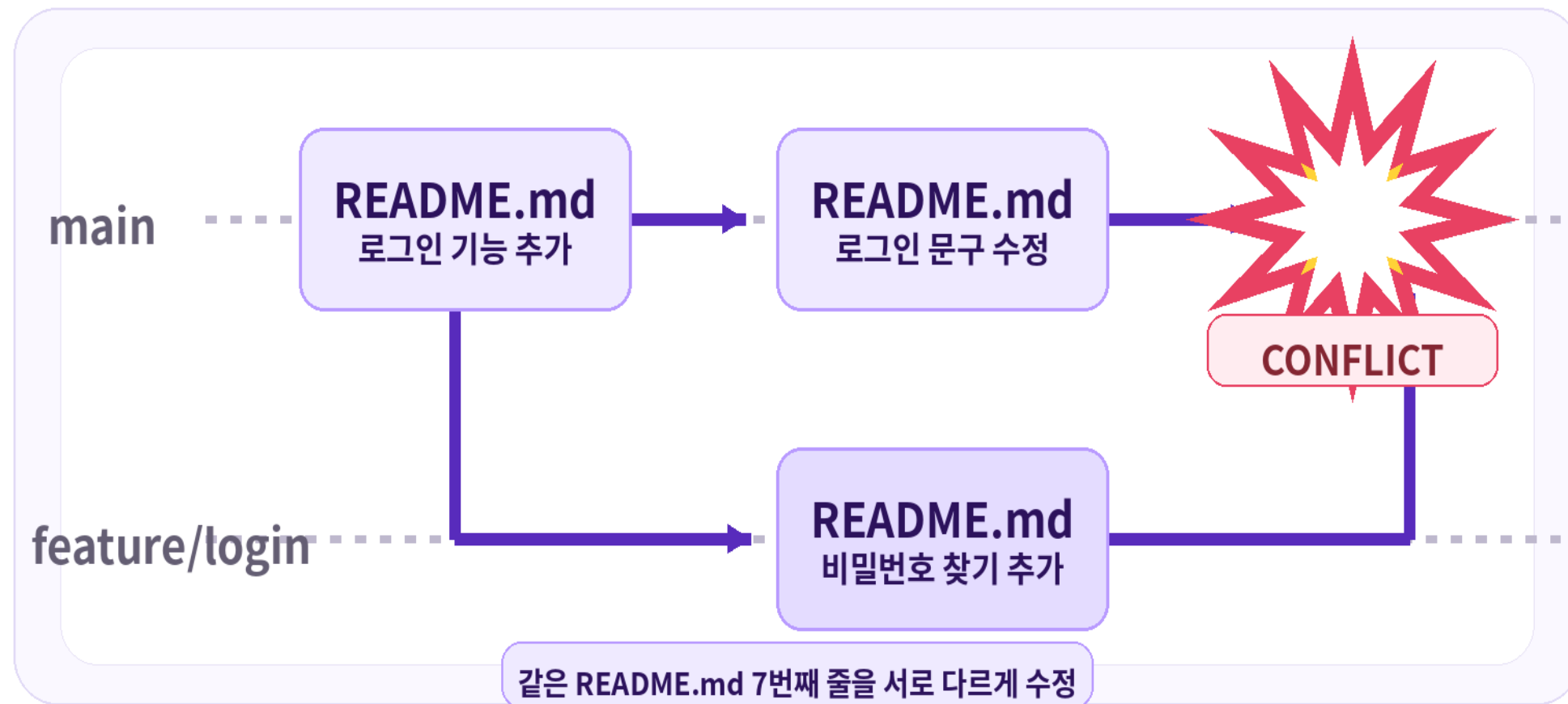
GitHub 협업

고급 기능과 문제 해결

충돌 해결하기

Merge Conflict : 충돌 발생

같은 파일의 같은 라인을 두 브랜치가 서로 다르게 수정하면, Git이 자동으로 합칠 수 없어 **사람에게 결정을 맡깁니다.**



1 같은 줄 동시 수정

두 사람이 같은 파일
같은 줄을 동시에 변경

2 같은 함수 리팩토링

같은 함수를 두 브랜치가
각자 다르게 정리

3 삭제 vs 수정

한쪽에서 삭제한 파일을
다른 쪽에서 수정

4 머지 작업 중

머지 작업 중
자동 병합 실패 ★ **가장 흔함**

VSCode에서의 충돌 마커

```
13
14 /**
15  * Prints the welcome message
16  */
17 Accept Current Change | Accept Incoming Change | Accept Both Changes | Compare Changes
18 <<<<<<< HEAD (Current Change)
19 function printMessage(showUsage, message) {
20     console.log(message);
21 }
22 =====
23 function printMessage(showUsage, showVersion) {
24     console.log("Welcome To Line Counter");
25     if (showVersion) {
26         console.log("Version: 1.0.0");
27     }
28 >>>>>>> theirs (Incoming Change)
29     if (showUsage) {
30         console.log("Usage: node base.js <file1> <file2> ... ");
31     }
32 }
33 /**
```

Git이 자동으로 파일에 삽입해주는 마커

<<<<<<< HEAD

충돌 시작 : 여기부터 아래 구분선
까지가 현재 브랜치(HEAD)의 코드

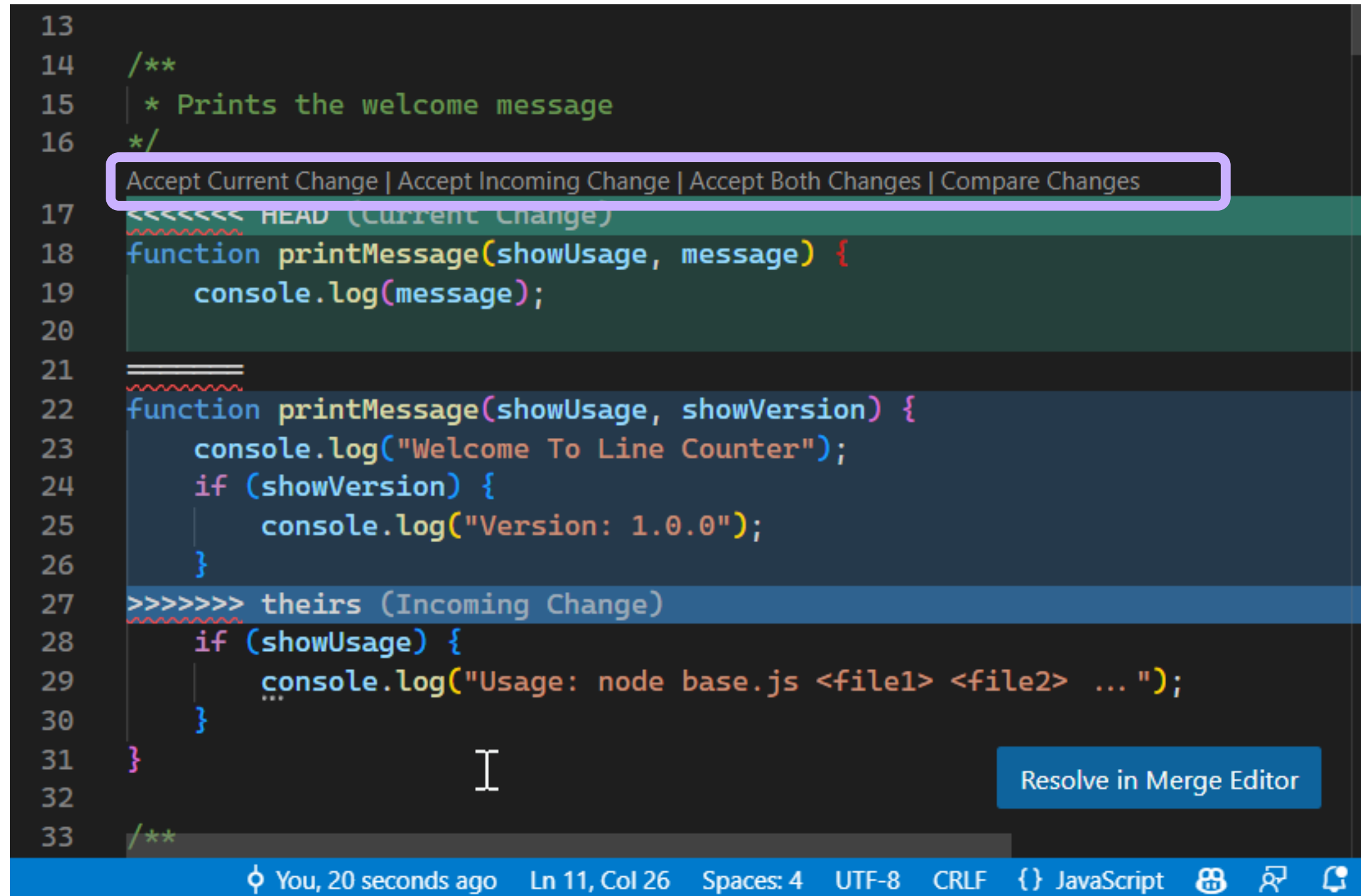
=====

구분선 : 위는 내 변경(HEAD), 아래는
가져온 변경(MERGE_HEAD)을 나눔

>>>>>>> branch-name

충돌 끝 : 병합하려는 상대 브랜치 이름이 표시되며,
여기까지가 충돌 블록임을 표시

VSCode에서의 충돌 액션 (4가지 옵션)



The screenshot shows a VS Code Merge Editor interface. A context menu is open over a conflict region, displaying four options: 'Accept Current Change', 'Accept Incoming Change', 'Accept Both Changes', and 'Compare Changes'. The code being merged is a JavaScript file named 'base.js'. The conflict is between the 'HEAD' (Current Change) and 'theirs' (Incoming Change). The 'HEAD' version has a function 'printMessage' that logs a message. The 'theirs' version has a function 'printMessage' that logs 'Welcome To Line Counter' and 'Version: 1.0.0'. The 'Accept Both Changes' option is highlighted in the context menu. A 'Resolve in Merge Editor' button is visible at the bottom right of the editor.

```
13  
14 /**  
15  * Prints the welcome message  
16  */  
17 <----- HEAD (Current Change)  
18 function printMessage(showUsage, message) {  
19     console.log(message);  
20 }  
21  
22 <===== theirs (Incoming Change)  
23 function printMessage(showUsage, showVersion) {  
24     console.log("Welcome To Line Counter");  
25     if (showVersion) {  
26         console.log("Version: 1.0.0");  
27     }  
28 }  
29  
30 >>>>>> theirs (Incoming Change)  
31 if (showUsage) {  
32     console.log("Usage: node base.js <file1> <file2> ... ");  
33 }  
34  
35 /**
```

Accept Current Change

내 변경(HEAD)만 살리고 가져온 변경은 버립니다.

Accept Incoming Change

가져온 변경(MERGE_HEAD)만 살리고 내 변경은 버립니다.

Accept Both Changes

양쪽 변경을 모두 남깁니다. 순서·중복은 직접 다듬어야 합니다.

Compare Changes

좌우 분할 비교 뷰를 열어 차이를 직접 확인하며 편집합니다.

VSCode에서의 3-way merge editor

JS target.js !

JS Merging: target.js !

merge-git-playground > JS target.js > printMessage

Incoming 7b18bdb • theirs

13

14 /**

15 * Prints the welcome message

16 */

Accept Incoming | Accept Combination (Incoming First) | Ignore

17 function printMessage(showUsage, showVersion) {

18 console.log("Welcome To Line Counter");

19 if (showVersion) {

20 console.log("Version: 1.0.0");

21 }

22 if (showUsage) {

23 console.log("Usage: node base.js <file1>");

24 }

Result merge-git-playground\target.js

Current b71d9b1 • main

13

14 /**

15 * Prints the welcome message

16 */

Accept Current | Accept Combination (Current First) | Ignore

17 function printMessage(showUsage, message) {

18 console.log(message);

19

20 if (showUsage) {

21 console.log("Usage: node base.js <file1>");

22 }

1 Conflict Remaining

13

14 /**

15 * Prints the welcome message

16 */

No Changes Accepted

17 function printMessage(showUsage) {

18 console.log("Welcome To Line Counter");

19

20 if (showUsage) {

21 console.log("Usage: node base.js <file1> <file2> ...");

22 }

Complete Merge

자주 하는 실수

```
VSCode - TERMINAL
$ git switch other-branch
error: Your local changes to the following files
would be overwritten by checkout:
    README.md
Please commit your changes or stash them before
you switch branches.
```

① 커밋하고 전환

```
$ git add .
$ git commit -m "wip: 작업 중"
$ git switch other-branch
```

✓ 정식 커밋으로 기록 (히스토리 남음)

브랜치 전환 전엔 항상 "지금 워킹트리가 깔끔한가?" 확인

→ **git status** (또는 VSCode 좌측 Source Control 패널)

작업한 변경이 아직 **커밋되지 않은 채**로 다른 브랜치로 이동하면 어떻게 될까?

→ 변경사항이 덮어써질 위험이 있으면 Git이 거부함

② 임시 보관(stash)하고 전환

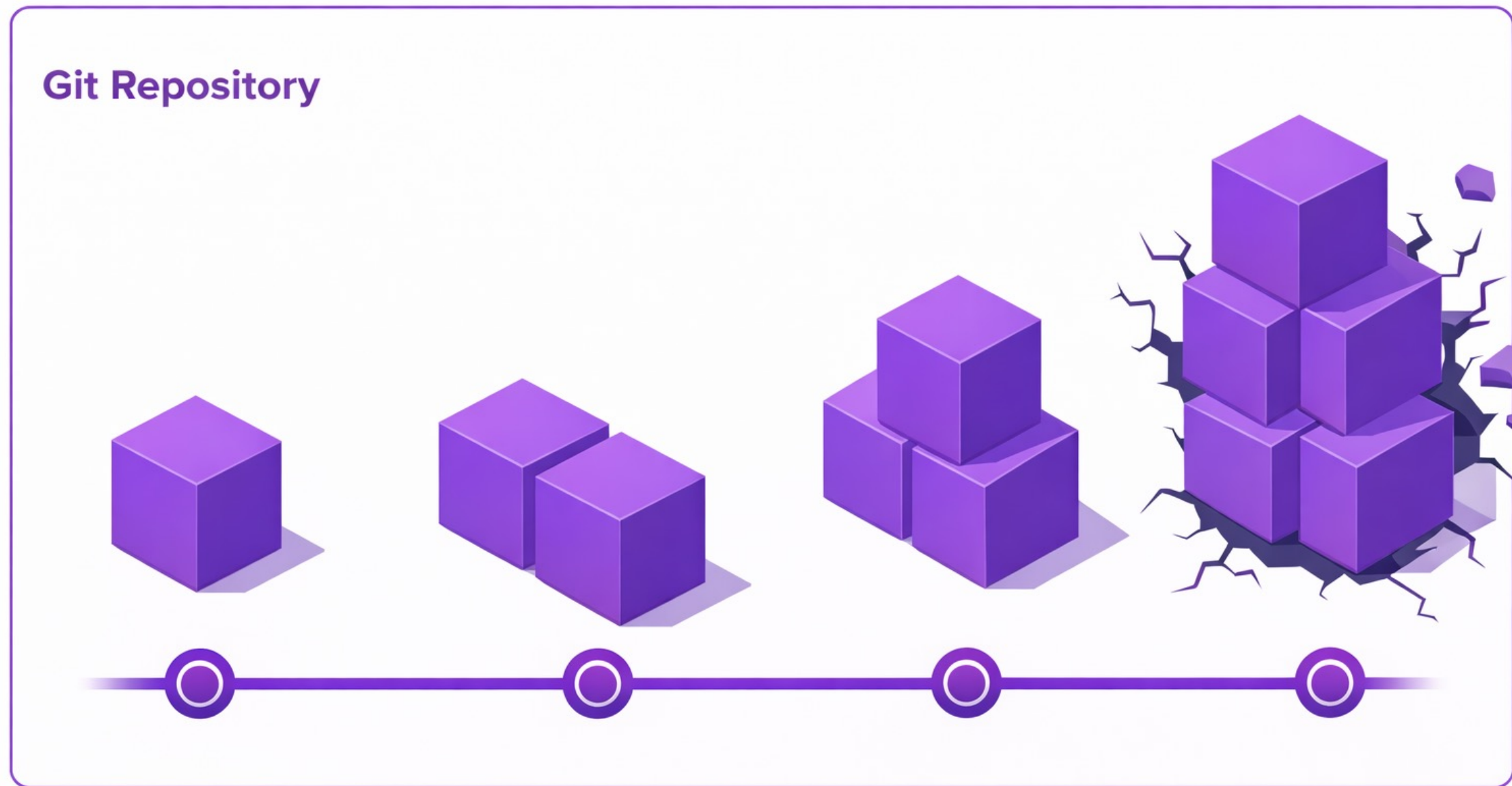
```
$ git stash
$ git switch other-branch
# 다시 돌아왔을 때
$ git stash pop
```

✓ 커밋 없이 임시 보관, 가볍게 처리

Git LFS 개념 및 활용 방법



기존 Git의 구조적 한계 : Snapshot



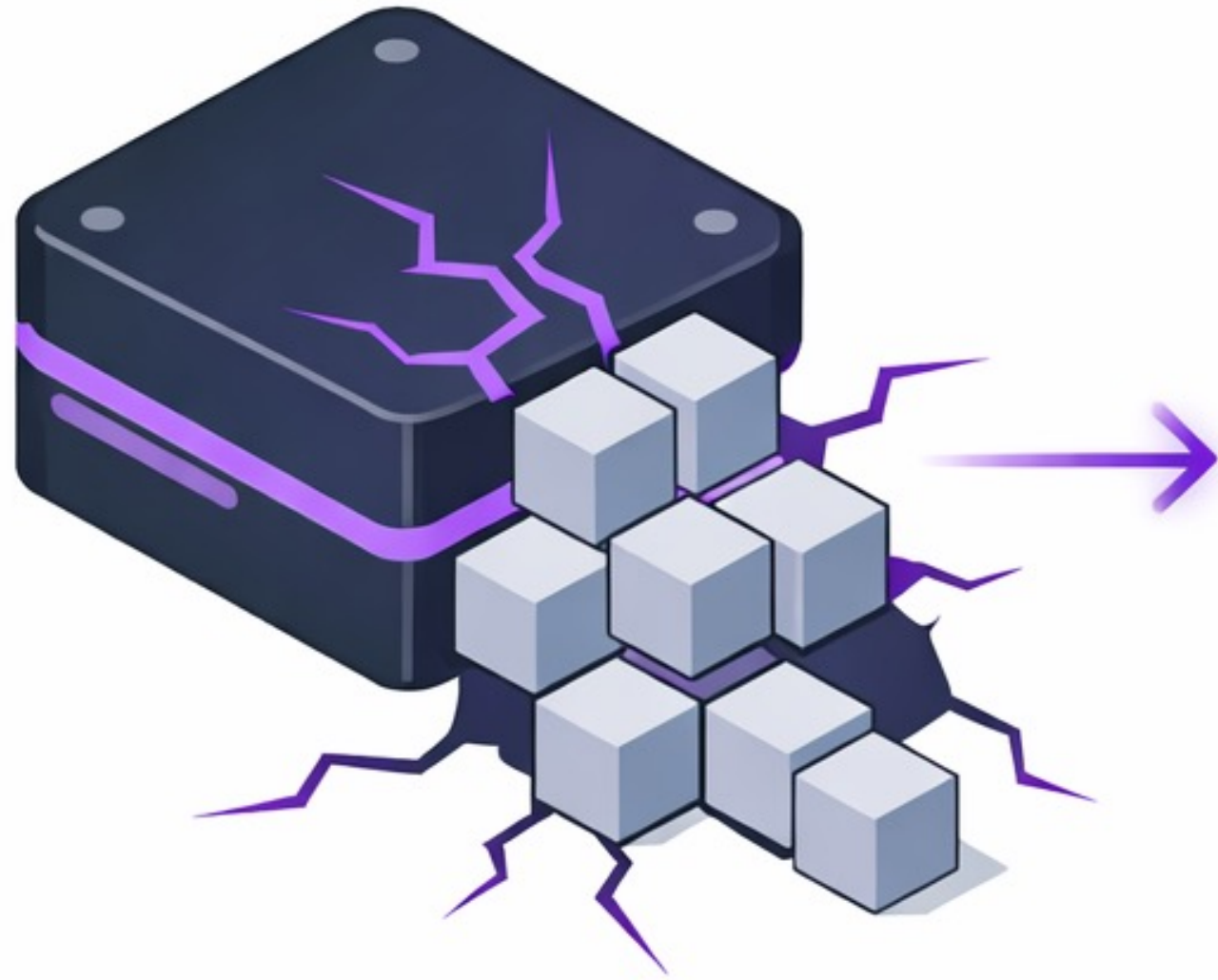
Snapshot 방식

Git은 커밋마다 전체 트리를 **스냅샷**으로 기록하지만, 변경되지 않은 파일은 **기존 객체를 그대로 참조**하므로 텍스트 파일은 효율적으로 관리됨

단, **이미지·영상·바이너리**처럼 **diff 효율이 떨어지는 파일**은 작은 수정에도 거의 전체가 새 객체로 저장 → **매 커밋마다 사실상 복제**되어 저장소 용량이 폭증



기존 Git의 구조적 한계 : Snapshot



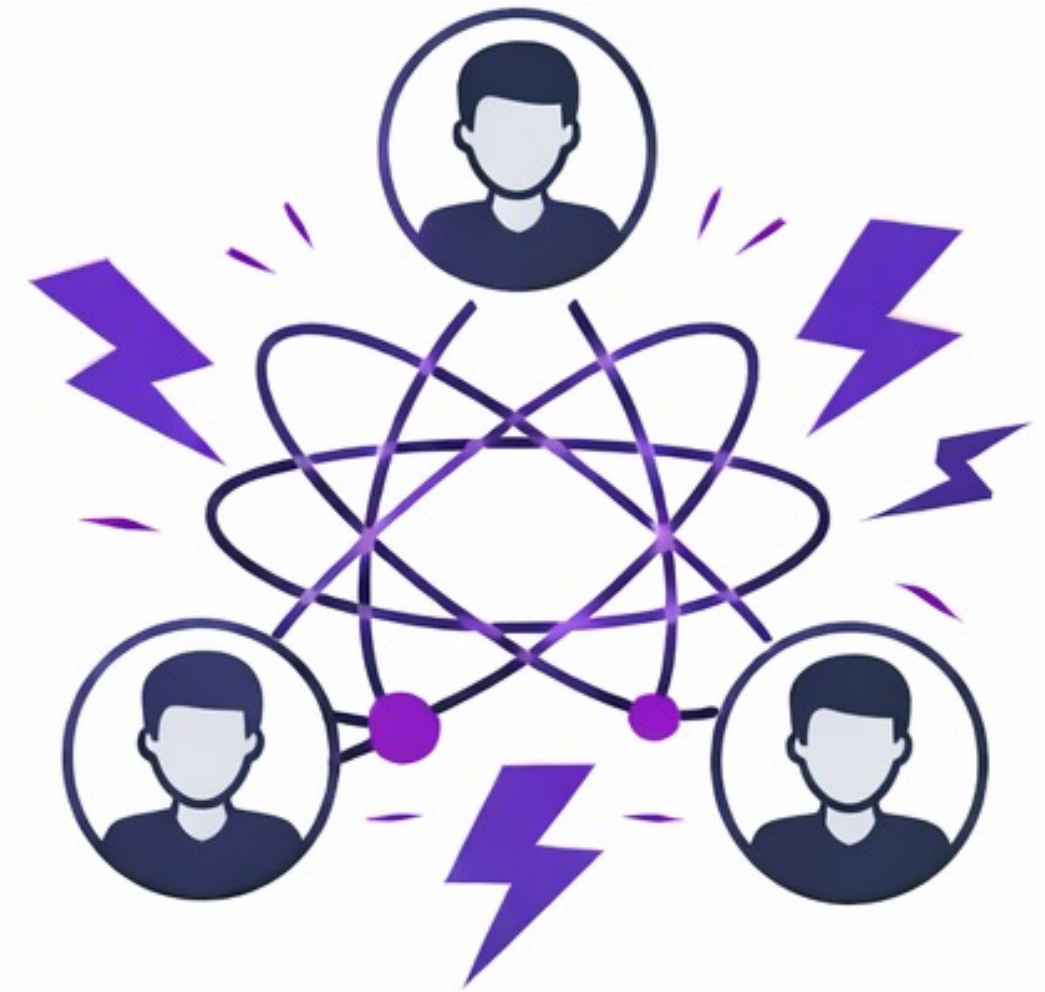
저장소 용량 급증

단일 대용량 파일 하나가 전체 레포지토리 크기를 압도합니다.



작업 속도 저하

파일 크기에 비례하여 Commit, Push, Clone에 소요되는 시간이 급격히 늘어납니다.

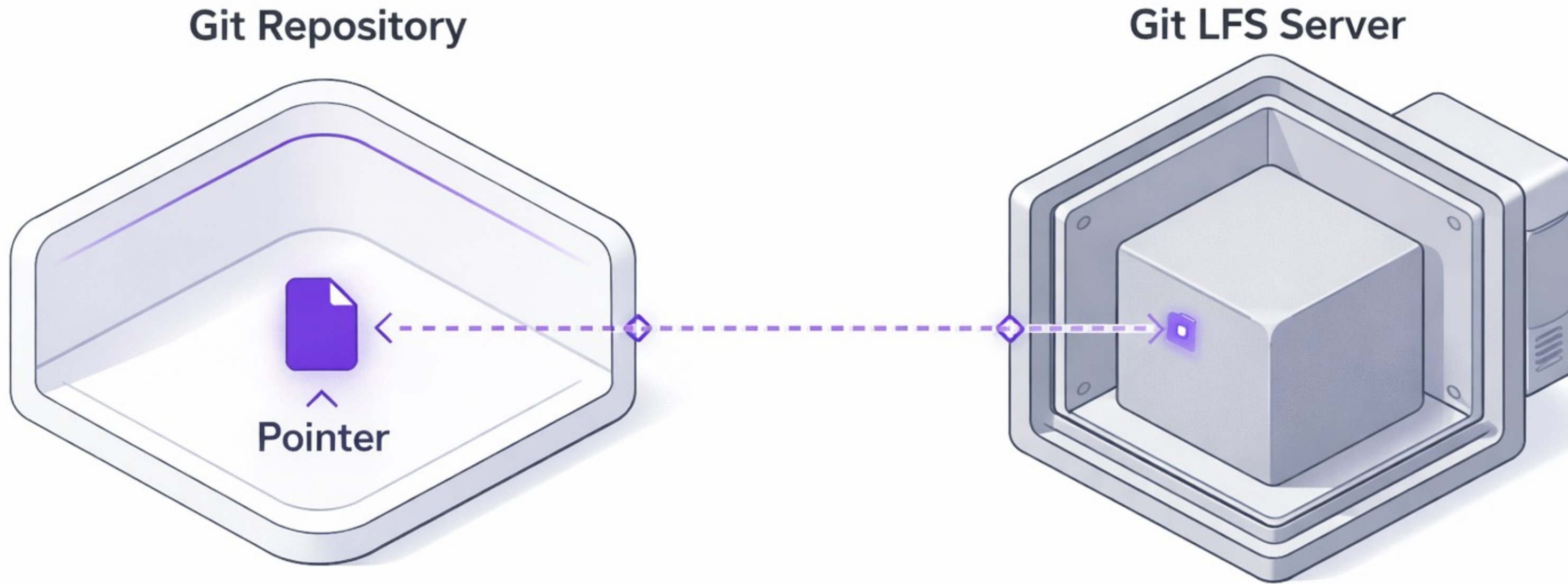


불필요한 히스토리 강제 공유

협업 시 모든 팀원이 원치 않는 대용량 파일의 변경 이력까지 강제로 다운로드해야 합니다.



Git LFS 개념 및 활용 방법



Git LFS(Large File Storage)

Git이 효율적으로 처리하지 못하는 대용량 파일을 다루기 위한 공식 확장 도구

외부 격리 저장

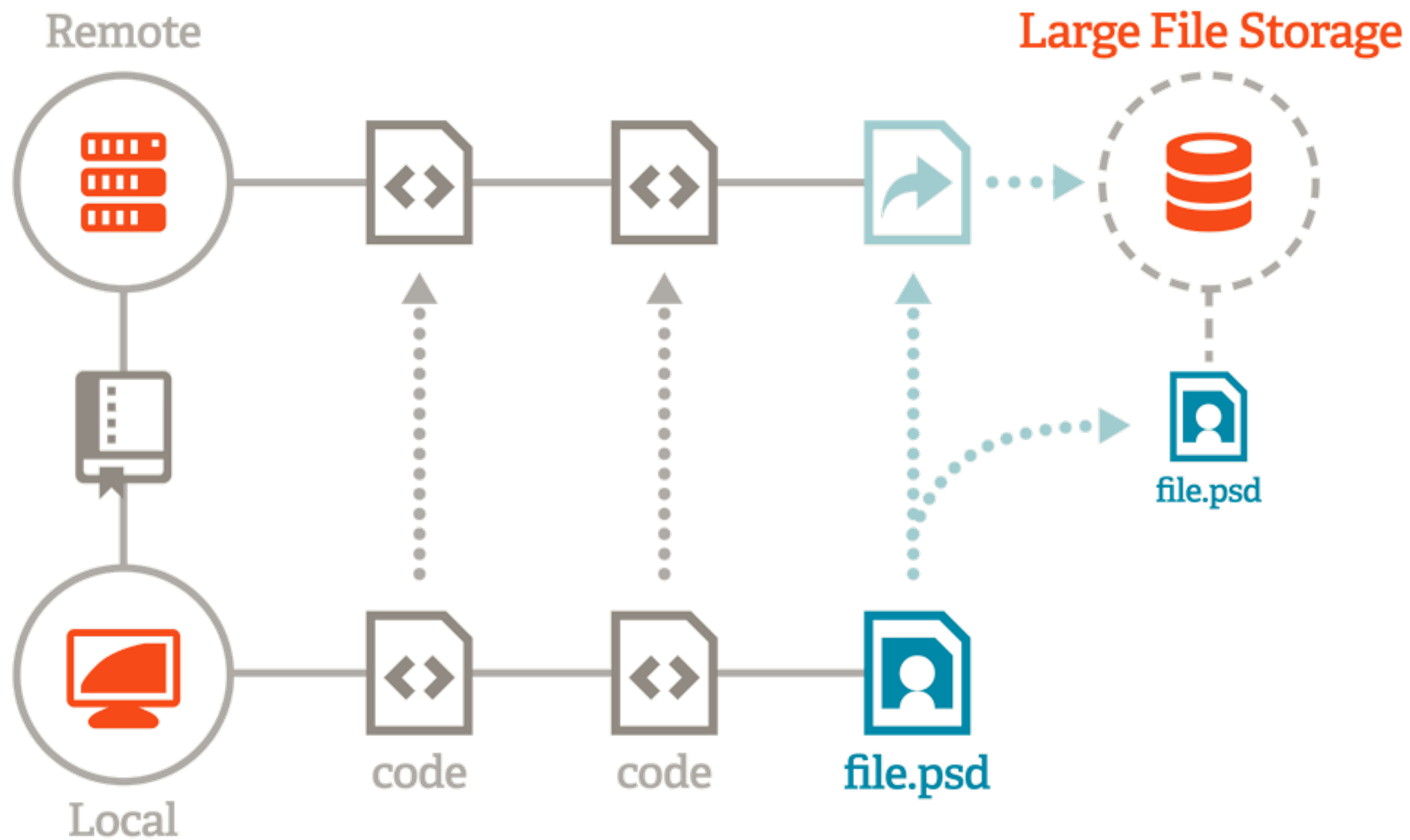
실제 대용량 파일은 Git 저장소 외부에 별도로 보관

포인터(Pointer) 파일

Git 내부 히스토리에는 원본 파일의 위치와 해시값만 담긴 수십 바이트 수준의 가벼운 포인터 파일만 저장됨



Git LFS 동작 구조



로컬(Local) : 개발자는 코드와 함께 대용량 파일의 참조 정보만 Git에 커밋함

리모트(Remote) : 원격 Git 서버는 참조만, 실제 대용량 파일은 LFS 서버에 저장



Git LFS 설치 (OS별)

운영체제	설치 명령어	초기화 / 비고
Mac OS	<code>brew install git-lfs</code>	<code>git lfs install</code> (사용자 환경 초기화 - 한 번만)
Windows	<code>choco install git-lfs</code> 또는 <code>winget install git-lfs</code>	<code>git lfs install</code>
Linux(Debian/Ubuntu)	<code>sudo apt-get update</code> <code>sudo apt-get install git-lfs</code>	<code>git lfs install</code>



Git LFS 특징

- 예를 들면, 게임 개발 프로젝트에서 고해상도 텍스처나 3D 모델과 같은 큰 파일을 관리해야 할 때,
- 프로젝트 저장소 크기를 줄이고, 파일을 다운로드하고 업데이트하는 데 걸리는 시간을 단축할 수 있음

대용량 파일 버전 관리	파일 크기에 제한 없이 대용량 파일을 버전 관리할 수 있도록 해주며 GB 단위의 파일도 문제없이 관리할 수 있음
저장소 크기 감소	파일의 포인터만 저장되므로 저장소 크기를 줄일 수 있으며 복제하거나 전송하는 데 걸리는 시간이 단축됨
Git 워크플로우 유지	Git의 기본 워크플로우를 유지하면서 대용량 파일을 관리할 수 있도록 해주며 파일 관리가 투명하게 이루어짐
다양한 호스팅 지원	GitHub, GitLab, Bitbucket 등 다양한 Git 호스팅 서비스에서 지원



Git LFS 대안 비교

- Git LFS : 대용량 파일을 포인터로 관리하여 저장소에 포함되지 않고 사용이 편리하며 협업에 적합함
- Git Submodule : 대용량 파일을 지원하지 않고 파일은 저장소에 포함되며 사용성과 협업 적합도가 낮음
- Git-annex : 대용량 파일을 저장소 외부에 보관하고 사용법이 복잡하며 협업에 적합하지 않음

기능	Git LFS	Git Submodule	Git-annex
대용량 파일 지원	Y	X	Y
히스토리 단순성	좋은(포인터)	보통	보통
사용 난이도	쉬움	중간	어려움
협업 적합도	높음	중간	낮음 (복잡함)

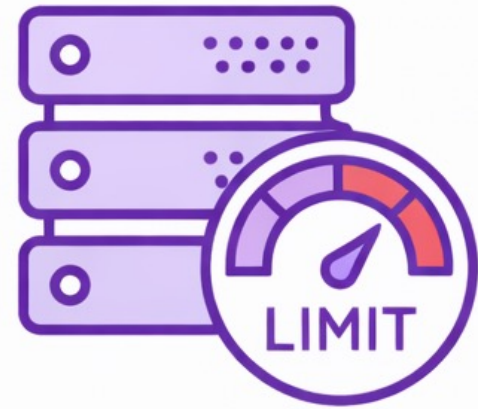


Git LFS 실무 활용 정리

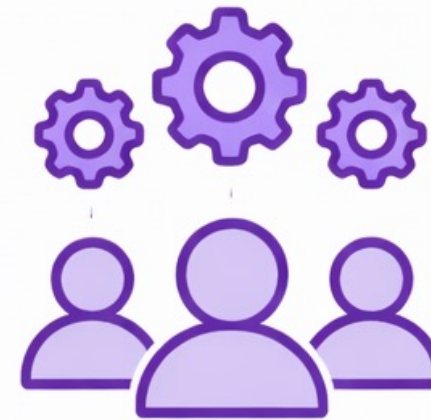
대용량 파일 버전 관리	.mp4, .psd, .zip, .csv 등 Git이 비효율적인 파일을 LFS로 추적
포인터 기반 저장	Git에는 포인터 파일만 저장되고, 파일은 LFS 서버에 저장됨
협업 시 자동 동기화	git pull 시 포인터 기반으로 자동으로 대용량 파일도 함께 내려받음
빠른 저장소 클론/푸시	파일을 나중에 개별 다운로드함으로써 속도 개선
CI/CD 파이프라인 연동	LFS 파일을 자동으로 받아와 테스트/배포에 활용 가능(git lfs pull)
LFS 파일만 개별 다운로드 가능	git lfs fetch, git lfs checkout으로 특정 파일만 받아올 수 있음
포인터 추적 확인	git lfs ls-files, .gitattributes로 추적 상태 점검 가능
저장소 용량 절약 및 관리	불필요한 캐시 정리(git lfs prune), 사용량 확인 가능
주의사항	공동 작업자 모두 git lfs install 필요, LFS 미지원 서버 주의



Git LFS 사용 시 고려사항



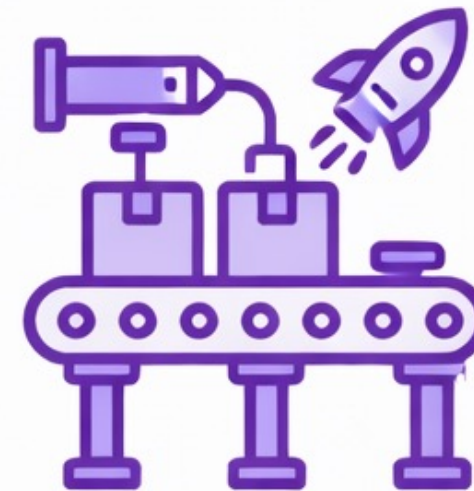
스토리지 및 트래픽 제한



전사적 환경 구성



호스팅 서버 지원 여부



CI/CD 파이프라인 연동

고급 Git 기능



고급 Git 기능



git stash

작업 중인 변경사항을
임시로 저장하고 복원



git cherry-pick

특정 커밋만 골라서
다른 브랜치에 적용



git rebase

커밋 히스토리를
깔끔하게 정리



git reflog

모든 이력 기록 보기
(삭제된 커밋도 복구 가능)



git bisect

이진 탐색으로
버그 원인 커밋 추적

Git Stash : 임시 저장 기능

- 작업 중이던 변경 사항을 임시로 저장하고 브랜치를 이동할 수 있음
- 브랜치 전환 시 미완성 작업 보관

명령어	설명
git stash	변경 사항 임시 저장
git stash list	저장된 stash 목록 보기
git stash apply	마지막 stash 적용 (삭제 안됨)
git stash pop	적용 + 목록에서 제거

```

~/test-project on develop +1
└─> git stash
Saved working directory and index state WIP on develop: cbfc661 chore: add LFS attributes

~/test-project on develop *1
└─> git stash list
stash@{0}: WIP on develop: cbfc661 chore: add LFS attributes

~/test-project on develop *1
└─> git stash apply
현재 브랜치 develop
커밋할 변경 사항:
  (use "git restore --staged <file>..." to unstage)
      새 파일:      test.py

└─> git stash pop
현재 브랜치 develop
커밋할 변경 사항:
  (use "git restore --staged <file>..." to unstage)
      새 파일:      test.py

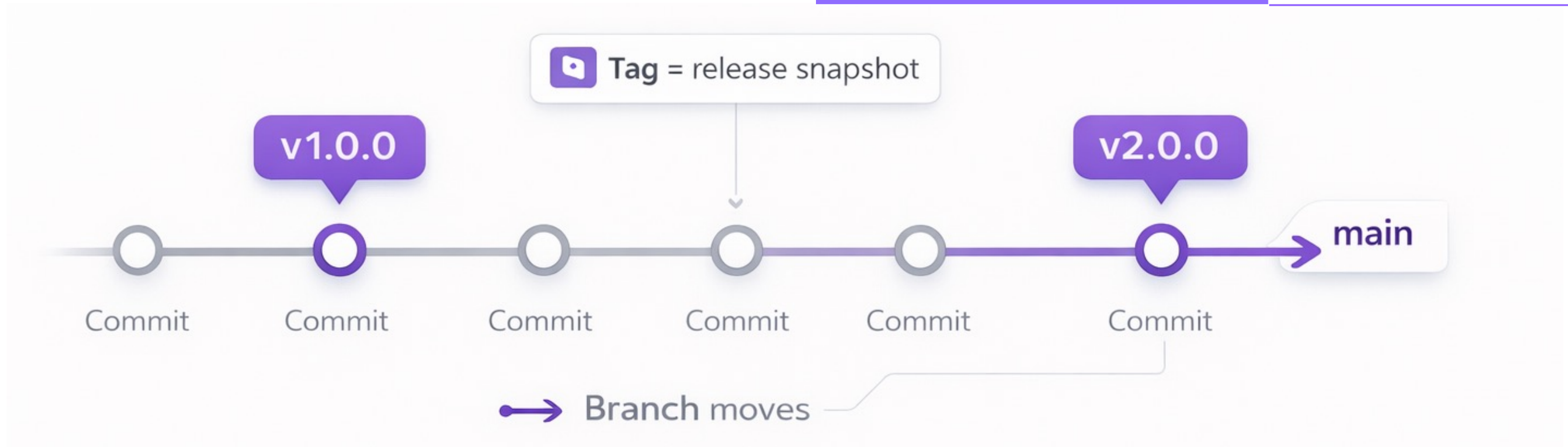
Dropped refs/stash@{0} (d75d6fc8702e50d1171b7dcceb2a549157c0d97e)

```

Git Tag : 버전 식별자 부여

- 커밋에 버전 태그를 지정해 릴리즈 포인트를 명확히 함
- 주로 main 브랜치의 릴리즈 커밋에 부여
- CI/CD 배포 시점 명확히 구분 및 버전 관리

명령어	설명
<code>git tag v1.0.0</code>	로컬에 태그 추가
<code>git push origin v1.0.0</code>	원격에 푸시





Git Cherry-pick : 선택적 커밋 적용

- 특정 커밋만 골라 다른 브랜치에 반영할 수 있음
- Hotfix, 긴급 수정 사항을 다른 브랜치에 빠르게 반영할 때 유용

명령어	설명
git cherry-pick	긴급 패치 커밋을 릴리즈 브랜치에 적용

```

~/test-project on develop +1
> git cherry-pick
사용법 : git cherry-pick [--edit] [-n] [-m <parent-number>] [-s] [-x] [--ff]
        [-S[<keyid>]] <commit>...

또는 : git cherry-pick (--continue | --skip | --abort | --quit)

--quit                되돌리기 또는 커밋 빼오기 연속을 끝냅니다
--continue            되돌리기 또는 커밋 빼오기 연속을 계속합니다
--abort               되돌리기 또는 커밋 빼오기 연속을 취소합니다
--skip                skip current commit and continue
--[no-]cleanup <모드> 메시지에서 공백과 #주석을 지웁니다
-n, --no-commit       자동으로 커밋하지 않습니다
--commit              opposite of --no-commit
-e, --[no-]edit       커밋 메시지를 편집합니다
-s, --[no-]signoff    add a Signed-off-by trailer
-m, --[no-]mainline <이전 -커밋 -번호>
                     이전 커밋 중 메인라인 선택
--[no-]rerere-autoupdate
                     가능하면 인덱스를 재사용한 충돌 해결로 업데이트합니다
--[no-]strategy <전략>
                     병합 전략
-X, --[no-]strategy-option <옵션>
                     병합 전략 옵션
-S, --[no-]gpg-sign[=<키 -ID>]
                     GPG 서명 커밋
-x                   커밋 이름을 뒤에 붙입니다
--[no-]ff             정방향 진행을 허용합니다
--[no-]allow-empty    최초 빈 커밋을 유지합니다
--[no-]allow-empty-message
                     빈 메시지로 커밋을 허용합니다
--[no-]keep-redundant-commits
                     deprecated: use --empty=keep instead
--empty (stop|drop|keep)
                     how to handle commits that become empty

```


Git Reflog : 과거 HEAD 추적

- 브랜치, HEAD, 리셋 등 Git 내부 이동 기록 추적 가능
- 실수로 브랜치 삭제, reset 등 했을 때 복구에 유용

명령어	설명
git reflog	실수로 지운 커밋 / 브랜치 복구

```
~/test-project on develop +1 x HUP ^ system
> git reflog
cbfc661 (HEAD -> develop, origin/main, origin/HEAD, main) HEAD@{0}: reset: moving to HEAD
cbfc661 (HEAD -> develop, origin/main, origin/HEAD, main) HEAD@{1}: checkout: moving from main to develop
cbfc661 (HEAD -> develop, origin/main, origin/HEAD, main) HEAD@{2}: commit: chore: add LFS attributes
cf53ef2 HEAD@{3}: commit: Add: big files via LFS
d9c5bf3 HEAD@{4}: commit: Add: initial commit
416a588 HEAD@{5}: clone: from http://127.0.0.1:58443/root/test-project.git
```



Git Bisect : 버그 발생 지점 추적

- 이진 탐색 방식으로 버그가 발생한 커밋을 추적
- 실수로 브랜치 삭제, reset 등 했을 때 복구에 유용

명령어	설명
git bisect start	대규모 프로젝트에서 특정 커밋 원인 추적

```
~/test-project on @cf53ef2b bisect +1
> git bisect start
A      test.py
이전 HEAD 위치는 cf53ef2 Add: big files via LFS
'develop' 브랜치로 전환합니다
status: waiting for both good and bad commits

~/test-project on @cf53ef2b bisect +1
> git bisect good
cbfc6619af7893f0d667201648cc197b5c719636 is the first bad commit
commit cbfc6619af7893f0d667201648cc197b5c719636 (origin/main, origin/HEAD, main, develop)
Author: cobyoo <shyeon.yoo@gmail.com>
Date:   Fri Aug 8 18:58:42 2025 +0900

    chore: add LFS attributes

.gitattributes | 2 ++
1 file changed, 2 insertions(+)
create mode 100644 .gitattributes
```